

Mantle Network

Security Assessment

CertiK Assessed on Mar 26th, 2026





Certik Assessed on Mar 26th, 2026

Mantle Network

The security assessment was prepared by Certik.

Executive Summary

TYPES

Layer 2

ECOSYSTEM

Mantle (MNT)

METHODS

Manual Review

LANGUAGE

Go, Rust, Solidity

TIMELINE

Preliminary comments published on 03/09/2026

Final report published on 04/02/2026

Vulnerability Summary



34

Total Findings

29

Resolved

0

Partially Resolved

5

Acknowledged

0

Declined

0 Centralization

Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

1 Major

1 Resolved



Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

13 Medium

11 Resolved, 2 Acknowledged



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

11 Minor

11 Resolved



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

9 Informational

6 Resolved, 3 Acknowledged



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | MANTLE NETWORK

Audit Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

System Overview

Review Notes

Findings

[MAN-31 : Unresettable BlobSourceChanged State In OpenData May Cause Sync Failure](#)

[MAN-01 : Core Dependencies Expose Known Vulnerabilities](#)

[MAN-02 : Use Of Wall Clock For Fork Gating May Introduce Operational Non-Determinism](#)

[MAN-18 : Channel Crossing Arsia Boundary Can Stall Safe-Head Progression](#)

[MAN-22 : Go/Rust Derivation Mismatch On Malformed Blob Decode Handling](#)

[MAN-25 : Potential Divide By Zero During Base-Fee Verification](#)

[MAN-27 : Mantle RLP Decode Path Does Not Enforce Full Payload Consumption](#)

[MAN-30 : Unused DA Limits In Preconf Environment](#)

[MAN-32 : Token Ratio Scaling Can Underflow Optimism Gas Limit Pre-Arsia](#)

[MAN-33 : `decode\\_deposit\(\)` Truncates `ethTxValue` And Creates Go/Rust Deposit Execution Divergence](#)

[MAN-35 : Duplicate `getBlobsV2` Hit Metric Increment](#)

[MAN-36 : Inconsistency Between Post-Arsia Decoding Behavior Between Kona And Op-Node.](#)

[MAN-37 : Failed OP Deposit Mints `BVM\\_ETH` After Commit, Leaving Journal And Warm-Cache Dirty For Next Tx](#)

[MAN-38 : Early Ecotone/Isthmus Activation In Rust Creates Go/Rust Divergence](#)

[MAN-03 : KMS Signer Client Created But Never Closed](#)

[MAN-05 : Missing Explicit Low-S Canonicalization In KMS Signature Conversion](#)

[MAN-07 : `trustrpc` Backup-Sync Flag Type Mismatch](#)

[MAN-17 : Gas Limit Setter Enforces Minimum But No Explicit Maximum Bound](#)

[MAN-19 : Potential Nil-Dereference Crash Paths](#)

[MAN-23 : `eth\\_estimateTotalFee` Does Not Preserve `blockHash` Selector Semantics](#)

[MAN-24 : Unchecked `uint256` `tokenRatio` Is Narrowed To Fixed-Width Gas Arithmetic, Producing Wrap-Vs-Fail-Stop Divergence](#)

[MAN-29 : Potential Nil Pointer Dereference In `makeEnv`](#)

[MAN-34 : Potential Panic Of CalcBaseFee On Missing BlobGasUsed](#)

[MAN-39 : Unbounded `tokenRatio` Configuration Can Trigger Runtime Panics In OP-REVM Fee Logic](#)

[MAN-40 : Panic DoS When RPC Code Bytes Start With EIP-7702 Magic But Are Not Valid EIP-7702](#)

[MAN-06 : Arsia Activation Path Lacks Runtime Verification That All Upgrade Transactions Succeeded](#)

[MAN-08 : Unclear Panic Message](#)

[MAN-09 : Usage Of Deprecated Types](#)

[MAN-10 : Commented Out Code](#)

[MAN-12 : Deposit Log Parser Uses Hardcoded Byte Layout Arithmetic With Stale Inline Spec Comments](#)

[MAN-13 : Unbounded `StorageKey` Decode Can Increase Memory Pressure On Proof Parsing Paths](#)

[MAN-20 : NatSpec Semver Mismatch In OperatorFeeVault](#)

[MAN-21 : Unused Return Value In DeriveL1GasInfoMantle](#)

[MAN-41 : Mantle Arsia DA Footprint Accounting Can Panic On Uninitialized `BlobGasUsed`](#)

Appendix

Disclaimer

CODEBASE | MANTLE NETWORK

Repository

<https://github.com/mantle-xyz/revm/commit/677049154af85305fde259f7bd7d291e9f553a30>

<https://github.com/mantle-xyz/kona/commit/83b0b6113028ec80ff30b5a9cc2585e3479a3fba>

<https://github.com/mantlenetworkio/op-geth/commit/c927a1bac55308bc157ccc2bc9e2360064d685b6>

<https://github.com/mantlenetworkio/mantle-v2/commit/6ad2298f39a7bf7e6f7ad5bb553a97ba32a5c757>

Commit

[677049154af85305fde259f7bd7d291e9f553a30](https://github.com/mantle-xyz/revm/commit/677049154af85305fde259f7bd7d291e9f553a30)

[83b0b6113028ec80ff30b5a9cc2585e3479a3fba](https://github.com/mantle-xyz/kona/commit/83b0b6113028ec80ff30b5a9cc2585e3479a3fba)

[c927a1bac55308bc157ccc2bc9e2360064d685b6](https://github.com/mantlenetworkio/op-geth/commit/c927a1bac55308bc157ccc2bc9e2360064d685b6)

[6ad2298f39a7bf7e6f7ad5bb553a97ba32a5c757](https://github.com/mantlenetworkio/mantle-v2/commit/6ad2298f39a7bf7e6f7ad5bb553a97ba32a5c757)

Audit Scope

The file in scope is listed in the appendix.

APPROACH & METHODS | MANTLE NETWORK

This audit was conducted for Mantle Network to evaluate the security and correctness of the smart contracts associated with the Mantle Network project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

SYSTEM OVERVIEW | MANTLE NETWORK

Mantle is an Ethereum-based Layer 2 network built on top of the OP Stack, with custom extensions for its own execution, fee, and data-publication model. Mantle does not participate in the Superchain and follows its own upgrade and integration path.

Prior to this upgrade, Mantle's production codebase ("Limb") was based on Optimism at the Regolith level, with its own feature set on top (Bedrock, Regolith, BaseFee, Everest, Euboea, Skadi, Limb). The current upgrade ("Arsia") replaces that base entirely with upstream Optimism v1.16.3, activating all intermediate upstream hardforks (Canyon, Delta, Ecotone, Fjord, Granite, Holocene, Isthmus, Jovian) simultaneously under a single Arsia activation trigger. The development followed a replacement and re-integration strategy: the previous Limb code was fully replaced by upstream v1.16.3; then Mantle's historical features were re-introduced on top of that new base. The audit focuses on the custom integration logic and backward-compatibility work that Mantle added to the upstream base.

In this section, we briefly describe the part of the system included in this diff audit scope. For a detailed view of the files included in the audit, please review the `Scope` section of this report.

The diff audit scope covers four repositories, each addressed through a dedicated PR:

- `mantle-v2` : [PR #290](#) (commit `6ad2298f`): The integration layer, including `op-node` derivation pipeline, `op-batcher`, fork configuration, and the scoped L1/L2 contracts.
- `op-geth` : [PR #148](#) (commit `c927a1ba`): The Go execution client. Scoped changes cover transaction execution and gas accounting (`state_transition.go` , `state_processor.go`), L1 cost calculation (`rollup_cost.go`), receipt construction, txpool validation, and the EVM/miner integration for Arsia.
- `kona` : [PR #19](#) (commit `83b0b611`): The Rust derivation and proving stack. Scoped changes cover Mantle-specific data sources and blob decoding, hardfork definitions, genesis and system config wiring, and the proof executor/client entry points.
- `revm` : [PR #20](#) (commit `67704915`): The Rust execution layer. Scoped changes cover the Optimism handler, L1 block info and fee caching, BVM_ETH mint/transfer operations, Mantle-specific precompiles, fork spec definitions, and transaction validation.

Taken together, these define both the live production path (Go execution via `op-geth` , Go derivation via `op-node`) and the Rust-side equivalence path (`revm` execution, `kona` derivation) relevant to replay, proving, and dispute-related safety.

Data Availability Transition

Mantle is transitioning from an earlier external (alternative) DA design to Ethereum-native data publication via blobs. Pre-Arsia, Mantle used external DA with its own blob format and decoding logic. Post-Arsia, the system moves toward standard Ethereum blob posting. The derivation pipeline must handle both modes correctly, and the boundary between them is a primary source of risk for historical replay and Go/Rust equivalence.

REVIEW NOTES | MANTLE NETWORK

Scope Addition: `op-eth` `tokenRatio` Cache Fix (PR #152)

During the audit engagement, the Mantle team identified and reported a bug in the `tokenRatio` caching logic within `op-eth`'s L1 cost functions (`NewL1CostFuncBeforeArsia` and `NewL1CostFuncArsia` in `core/types/rollup_cost.go`). The original implementation cached `tokenRatio` at closure creation time and applied a compare-and-swap mechanism intended to ensure that a transaction modifying `tokenRatio` would still be charged using the previous value. This caching logic was incorrect under certain state transition sequences. The Mantle team addressed the issue in [PR #152](#) (commit [1118230](#)). As for Mantle team's request, this PR was added to the audit scope. We reviewed the fix and confirmed that it correctly resolves the reported caching inconsistency.

Fork-Boundary Sensitivity

The scoped changes are concentrated around Mantle-specific hardfork behavior, especially Arsia and the blob and fee logic that comes with it. These boundaries are especially sensitive because configuration changes, contract-side parameter updates, derivation behavior, and execution behavior all shift at roughly the same time. A bug at a fork boundary can therefore appear in multiple forms at once, such as a fee-accounting mismatch in execution, a replay mismatch in derivation, or a Rust-vs-Go divergence that only appears in the transition window.

Out-of-Scope Dependencies

This audit did not re-audit all upstream OP Stack, geth, or general Rust and Go dependencies from scratch. Components outside the scoped PRs were treated as inherited dependencies unless Mantle-specific changes modified their behavior or assumptions. In particular:

- Many components of `op-eth` are inherited directly from go-ethereum. Components not explicitly listed in the audit scope were treated as black boxes and assumed to be correct as per upstream.
- External Rust crates and unchanged `kona/revm` modules were not audited line-by-line, under the assumption that they have been reviewed in their upstream context.
- The codebase contains legacy and currently inactive paths (e.g., pre-BVMETHMintUpgrade deposit handling, dormant precompile registrations) that are kept for historical sync compatibility. These paths were reviewed for safety at integration boundaries but were not the primary audit focus.

Our review focused on Mantle's live and proving behavior within the defined scope. Use of this codebase on custom chains, forks, or in areas not covered by the audit was not evaluated and should not be presumed secure from this assessment alone.

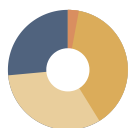
Testing and Documentation

The codebase includes helpful inline comments, architectural notes, and unit tests, which make it easy to understand and work with. To further strengthen the existing testing suite, it could be helpful to gradually expand end-to-end and parity-focused regression testing, especially to increase confidence in how different components interact. For example, this could include additional test cases around fork activation boundaries, same-block parameter changes, malformed inputs, and historical replay behavior, along with checks to confirm consistent results between the Go and Rust implementations under

the same conditions. These areas are particularly relevant in Mantle, where some of the more impactful risks relate to interactions across components rather than individual functions.

On the documentation side, the existing file-level comments are clear and useful. As a next step, it might also be helpful to have a higher-level document that outlines the expected end-to-end transaction flow across all four repositories for a given lifecycle, which could further support long-term maintenance and future audit work.

FINDINGS | MANTLE NETWORK



34

Total Findings

0

Critical

0

Centralization

1

Major

13

Medium

11

Minor

9

Informational

This report has been prepared for Mantle Network to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 34 issues were identified. Leveraging a combination of Manual Review the following findings were uncovered:

ID	Title	Category	Severity	Status
MAN-31	Unresettable BlobSourceChanged State In OpenData May Cause Sync Failure	Coding Issue	Major	● Resolved
MAN-01	Core Dependencies Expose Known Vulnerabilities	Denial of Service, Volatile Code	Medium	● Resolved
MAN-02	Use Of Wall Clock For Fork Gating May Introduce Operational Non-Determinism	Denial of Service	Medium	● Resolved
MAN-18	Channel Crossing Arsia Boundary Can Stall Safe-Head Progression	Design Issue	Medium	● Resolved
MAN-22	Go/Rust Derivation Mismatch On Malformed Blob Decode Handling	Inconsistency	Medium	● Resolved
MAN-25	Potential Divide By Zero During Base-Fee Verification	Logical Issue	Medium	● Resolved
MAN-27	Mantle RLP Decode Path Does Not Enforce Full Payload Consumption	Coding Issue	Medium	● Resolved
MAN-30	Unused DA Limits In Preconf Environment	Logical Issue	Medium	● Resolved
MAN-32	Token Ratio Scaling Can Underflow Optimism Gas Limit Pre-Arsia	Incorrect Calculation	Medium	● Acknowledged
MAN-33	<code>decode_deposit()</code> Truncates <code>ethTxValue</code> And Creates Go/Rust Deposit Execution Divergence	Logical Issue	Medium	● Acknowledged

ID	Title	Category	Severity	Status
MAN-35	Duplicate <code>getBlobsV2</code> Hit Metric Increment	Coding Issue	Medium	● Resolved
MAN-36	Inconsistency Between Post-Arsia Decoding Behavior Between Kona And Op-Node.	Logical Issue	Medium	● Resolved
MAN-37	Failed OP Deposit Mints <code>BVM_ETH</code> After Commit, Leaving Journal And Warm-Cache Dirty For Next Tx	Coding Issue	Medium	● Resolved
MAN-38	Early Ecotone/Isthmus Activation In Rust Creates Go/Rust Divergence	Logical Issue	Medium	● Resolved
MAN-03	KMS Signer Client Created But Never Closed	Volatile Code	Minor	● Resolved
MAN-05	Missing Explicit Low-S Canonicalization In KMS Signature Conversion	Logical Issue	Minor	● Resolved
MAN-07	<code>trustrpc</code> Backup-Sync Flag Type Mismatch	Inconsistency	Minor	● Resolved
MAN-17	Gas Limit Setter Enforces Minimum But No Explicit Maximum Bound	Coding Issue, Inconsistency	Minor	● Resolved
MAN-19	Potential Nil-Dereference Crash Paths	Coding Style	Minor	● Resolved
MAN-23	<code>eth_estimateTotalFee</code> Does Not Preserve <code>blockHash</code> Selector Semantics	Coding Issue, Volatile Code	Minor	● Resolved
MAN-24	Unchecked <code>uint256</code> <code>tokenRatio</code> Is Narrowed To Fixed-Width Gas Arithmetic, Producing Wrap-Vs-Fail-Stop Divergence	Inconsistency, Incorrect Calculation	Minor	● Resolved
MAN-29	Potential Nil Pointer Dereference In <code>makeEnv</code>	Logical Issue	Minor	● Resolved
MAN-34	Potential Panic Of CalcBaseFee On Missing BlobGasUsed	Coding Issue	Minor	● Resolved

ID	Title	Category	Severity	Status
MAN-39	Unbounded <code>tokenRatio</code> Configuration Can Trigger Runtime Panics In OP-REVM Fee Logic	Coding Style	Minor	Resolved
MAN-40	Panic DoS When RPC Code Bytes Start With EIP-7702 Magic But Are Not Valid EIP-7702	Logical Issue	Minor	Resolved
MAN-06	Arsia Activation Path Lacks Runtime Verification That All Upgrade Transactions Succeeded	Volatile Code, Inconsistency	Informational	Acknowledged
MAN-08	Unclear Panic Message	Inconsistency	Informational	Resolved
MAN-09	Usage Of Deprecated Types	Volatile Code	Informational	Resolved
MAN-10	Commented Out Code	Coding Style	Informational	Resolved
MAN-12	Deposit Log Parser Uses Hardcoded Byte Layout Arithmetic With Stale Inline Spec Comments	Coding Style	Informational	Acknowledged
MAN-13	Unbounded <code>StorageKey</code> Decode Can Increase Memory Pressure On Proof Parsing Paths	Denial of Service	Informational	Acknowledged
MAN-20	NatSpec Semver Mismatch In OperatorFeeVault	Inconsistency	Informational	Resolved
MAN-21	Unused Return Value In DeriveL1GasInfoMantle	Coding Style	Informational	Resolved
MAN-41	Mantle Arsia DA Footprint Accounting Can Panic On Uninitialized <code>BlobGasUsed</code>	Coding Issue	Informational	Resolved

MAN-31 | Unresettable BlobSourceChanged State In OpenData May Cause Sync Failure

Category	Severity	Location	Status
Coding Issue	● Major	op-node/rollup/derive/data_source.go (base-mantle-v2): 72~96	● Resolved

Description

The `DataSourceFactory.OpenData` function decides whether to use `calldata`, `MantleBlobDataSource`, or `BlobDataSource` based on `ecotoneTime`, `mantleEverestTime`, and a mutable boolean flag `blobSourceChanged` stored on the factory. When `NewMantleBlobDataSource` encounters a specific condition, it invokes a callback that permanently sets `ds.blobSourceChanged = true`. After this switch, for Ecotone active blocks `OpenData` always selects `NewBlobDataSource` whenever `blobSourceChanged` is true. There is no logic to reset or recompute `blobSourceChanged` during L1 chain reorganizations. If the L1 chain reorgs to a block height before the switch, the node must re-derive historical blocks that still require `MantleBlobDataSource`, but `OpenData` continues to return `BlobDataSource` due to the latched flag. As a result, historical Mantle formatted blobs are decoded with the standard blob decoder, leading to potential decoding failures, derivation pipeline stalls, and the node falling out of sync.

Proof of Concept

For reproducibility add this test into `mantle-v2/op-node/rollup/derive/data_source_poc_test.go` file.

```
package derive

import (
    "context"
    "math/big"
    "testing"

    "github.com/ethereum/go-ethereum/common"
    "github.com/ethereum/go-ethereum/log"
    "github.com/stretchr/testify/require"

    "github.com/ethereum-optimism/optimism/op-node/rollup"
    "github.com/ethereum-optimism/optimism/op-service/eth"
    "github.com/ethereum-optimism/optimism/op-service/testlog"
    "github.com/ethereum-optimism/optimism/op-service/testutils"
)

func TestOpenData_LatchedBlobSourceChangedOverridesHistoricalMantleSource(t
*testing.T) {
    logger := testlog.Logger(t, log.LvlInfo)
    ctx := context.Background()

    ecotoneTime := uint64(100)
    mantleEverestTime := uint64(50)
    batchInbox := common.Address{0x42}
    batcherAddr := common.Address{0x24}
    ref := eth.L1BlockRef{
        Hash:    common.HexToHash("0x1234"),
        Number: 10,
        Time:    ecotoneTime + 1,
    }

    cfg := &rollup.Config{
        L1ChainID:    big.NewInt(1),
        BatchInboxAddress: batchInbox,
        EcotoneTime:    &ecotoneTime,
        MantleEverestTime: &mantleEverestTime,
    }

    freshFactory := NewDataSourceFactory(
        logger,
        cfg,
        &testutils.MockL1Source{},
        &testutils.MockBlobsFetcher{},
        nil,
    )

    freshSrc, err := freshFactory.OpenData(ctx, ref, batcherAddr)
    require.NoError(t, err)
    require.IsType(
```

```

        t,
        &MantleBlobDataSource{},
        freshSrc,
        "the same historical ref should use MantleBlobDataSource when no later
toggle has latched",
    )

    latchedFactory := NewDataSourceFactory(
        logger,
        cfg,
        &testutils.MockL1Source{},
        &testutils.MockBlobsFetcher{},
        nil,
    )
    latchedFactory.blobSourceChanged = true

    latchedSrc, err := latchedFactory.OpenData(ctx, ref, batcherAddr)
    require.NoError(t, err)
    require.IsType(
        t,
        &BlobDataSource{},
        latchedSrc,
        "after blobSourceChanged is latched, the same historical ref is forced onto
the standard blob source",
    )
}

```

Recommendation

Refactor `OpenData` so that data source selection depends only on the current L1 origin (`ref` hash/number/timestamp) and fork configuration (`ecotoneTime` , `mantleEverestTime` , `Arsia`), and not on a latched `blobSourceChanged` flag; remove or ignore `blobSourceChanged` and always recompute the correct iterator (`NewCallldataSource` , `NewMantleBlobDataSource` , `NewBlobDataSource`) from the current origin. Alternatively, we recommend clarifying the intended behavior.

Alleviation

[Mantle Network , 03/12/2026]:

Issue acknowledged. L1 reorg around Arsia activation does indeed have a probability of causing node derivation pipeline to fail.

The recommended implementation could solve this issue but would cause another problem fixed by the `blobSourceChanged` toggle.

A fact is that we can never be certain of the block a tx sent will appear in. Then no matter how the batcher decides the blob encoding format (either based on an already-seen L1 block or the wall clock), there is still possibility that the submitted tx gets mined in an unexpected block. For example, batcher decides to use pre-Arsia format, but the tx accidentally appears in a post-Arsia block. That's why node cannot rely on L1 time to decode blobs.

And also that's why we introduce a toggle. It's designed to separate blob format change from Arsia activation. In fact, batcher could choose any time to perform a format change as long as this change is a one-time-only job. And node's job is detecting this change by a func that supports both formats in an infinite long time window starting from deployment. In this way, we could handle the above problem. The point is that how to make the time of performing format change not happen twice or even more.

Since we changed to use L1 block within batcher, and L1 reorg is possible, an appropriate fix to this issue is making data source a resettable stage and reset toggle to false when any kinds of reset happens.

Changes have been reflected in the commit hash: <https://github.com/mantlenetworkio/mantle-v2/commit/984e9abfb3d6a34f1d0387777ff9bc08eea53fd>

[CertiK , 03/12/2026]:

The team resolved the issue by making the data source resettable and clearing `blobSourceChanged` in `DataSourceFactory.Reset()`, so that after any derivation reset the next `OpenData` call re-selects the correct blob source from the current L1 origin, in commit [984e9ab](#).

MAN-01 | Core Dependencies Expose Known Vulnerabilities

Category	Severity	Location	Status
Denial of Service, Volatile Code	● Medium	go.mod (base-mantle-v2): 3~5, 18	● Resolved

Description

The file `go.mod` pins `go 1.24.0` / `toolchain go1.24.10`, and `govulncheck` reports reachable standard-library issues at this patch level (including `crypto/tls` and `crypto/x509` advisories fixed in newer 1.24.x patches). The same dependency graph also includes `github.com/consensys/gnark-crypto@v0.18.0`, with reachable `GO-2025-4087` traces (unchecked allocation during vector deserialization). Combined, these increase operational security risk in network-facing and parsing-heavy paths, primarily via DoS/resource-pressure and TLS/x509 hardening gaps.

Recommendation

We recommend standardizing audited repos on an updated Go patch baseline (at least `1.24.13+`, preferably latest stable at release time), upgrading `github.com/consensys/gnark-crypto` to `v0.18.1+`, and enforcing CI `govulncheck` gates. We also recommend re-running targeted regression tests after upgrades to verify both vulnerability removal and behavioral stability.

Alleviation

[Mantle Network , 02/25/2026]: Issue acknowledged. Changes have been reflected in the commit hash:
<https://github.com/mantlenetworkio/mantle-v2/commit/28c405dbd1680f68a05280c23846492e69a20edb>

[CertiK , 02/25/2026]: The team heeded the advice and resolved the issue by fixing the vulnerable dependencies in commit [28c405dbd1680f68a05280c23846492e69a20edb](https://github.com/mantlenetworkio/mantle-v2/commit/28c405dbd1680f68a05280c23846492e69a20edb)

MAN-02 | Use Of Wall Clock For Fork Gating May Introduce Operational Non-Determinism

Category	Severity	Location	Status
Denial of Service	● Medium	op-batcher/batcher/driver.go (base-mantle-v2): 1013; op-batcher/batcher/service.go (base-mantle-v2): 295~297; op-service/sources/backup_sync_client.go (base-mantle-v2): 104	● Resolved

Description

The batcher currently decides Arsia blob-format behavior from local wall-clock time (`time.Now().Unix()`), instead of deriving fork state from the chain context attached to the batch data. Around activation boundaries, even modest clock skew, restart timing, or active/passive failover drift can make operators choose different encoding branches. Operators handling the same backlog can choose different encodings near activation boundaries, introducing operational non-determinism. The primary impact is liveness and reliability risk including wasted L1 submission fees, rejected or ignored batch data, and safe-head lag.

Note that the same host-time gating pattern also appears in startup config checks (op-batcher/batcher/service.go), where impact is lower (initialization/warning behavior) but still contributes to operational inconsistency near fork boundaries. Also a lower-impact wall-clock dependency also exists in backup-sync open-ended range targeting (op-service/sources/backup_sync_client.go). In that path, local clock skew can change how far ahead the node schedules fetch requests, which can cause catch-up jitter, unnecessary retries, or temporary lag. This affects sync efficiency and liveness behavior, but payload validity is still enforced downstream by normal validation checks

Recommendation

We recommend replacing wall-clock-based fork gating with a chain-derived timestamp that is explicitly bound to the batch or channel being encoded, and propagating that timestamp into blob-format selection so the decision remains stable across retries and across operators. We also recommend defining a deterministic fallback for cancellation/empty-transaction paths, adding boundary-focused tests around Arsia activation (including restart, clock-skew, and active/passive failover scenarios), and adding observability for pre-/post-fork format selection to make drift detectable during operations.

Alleviation

[Mantle Network , 02/26/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: <https://github.com/mantlenetworkio/mantle-v2/commit/43846ee87a0d95b9ce5aa579a3e933a707d12e72> and <https://github.com/mantlenetworkio/mantle-v2/commit/0ea0682d12dd88ae599d7d5d37efe07666fddd3b>

[CertiK , 03/03/2026]: The team heeded the advice and resolved the use of wall clock risk issue by in commits [0ea0682d12dd88ae599d7d5d37efe07666fddd3b](https://github.com/mantlenetworkio/mantle-v2/commit/0ea0682d12dd88ae599d7d5d37efe07666fddd3b) and [43846ee87a0d95b9ce5aa579a3e933a707d12e72](https://github.com/mantlenetworkio/mantle-v2/commit/43846ee87a0d95b9ce5aa579a3e933a707d12e72).

MAN-18 Channel Crossing Arsia Boundary Can Stall Safe-Head Progression

Category	Severity	Location	Status
Design Issue	● Medium	op-batcher/batcher/driver.go (base-mantle-v2): 1010~1023; op-batcher/batcher/tx_data.go (base-mantle-v2): 12~59; op-node/rollup/derive/data_source.go (base-mantle-v2): 82~89	● Resolved

Description

The Arsia transition introduces a format split between pre-Arsia and post-Arsia blob handling, while batch submission format selection is time-dependent at submission. This creates an operationally sensitive boundary where mixed-format channel/frame handling can lead to frame drops and a risk of safe-head progression stalls. Because mitigation currently depends on deployment choreography (for example, stopping/draining the batcher) rather than strict protocol-level guardrails in `op-batcher`, missed rollout steps can increase derivation liveness risk during activation.

Recommendation

We recommend adding explicit batcher-side fork-boundary guardrails so mixed pre-/post-Arsia channels cannot be produced or clarifying the intended risk mitigation procedure.

Alleviation

[Mantle Network, 03/03/2026]:

The identified behavior is limited to the Arsia upgrade boundary window. As mitigation, we will enforce an operational guardrail to stop the batcher before activation and restart it after activation, and document this in the upgrade runbook. Given this is a time-bounded scenario and protocol-level derivation changes may introduce broader compatibility risk, we prefer operational control at this stage. Under the OP threat model, a fully malicious batcher can already cause wider disruption, so this specific vector is not treated as a standalone protocol-priority issue.

[CertiK, 03/03/2026]:

We acknowledge the team's rationale and agree that the issue is bounded to the Arsia activation window. The stop/restart batcher runbook control is an adequate mitigation for this scenario. On this basis and considering that after the fork upgrade the issue is not present anymore, the finding is considered resolved, with a recommendation to maintain clear documentation containing procedure steps, ownership, and execution checklist.

MAN-22 | Go/Rust Derivation Mismatch On Malformed Blob Decode Handling

Category	Severity	Location	Status
Inconsistency	● Medium	crates/protocol/derive/src/sources/mantle_blob.rs (base-kona): 203~210; op-node/rollup/derive/mantle_blob_source.go (base-mantle-v2): 183~202	● Resolved

Description

Malformed blob decoding does not follow the same policy in Go and Rust: the Go path may continue through fallback behavior while the Rust path can hard-fail. Because both implementations process the same L1 inputs, this difference creates a real parity hazard in edge cases. In practice, corrupted or malformed blob inputs can cause divergent derivation outcomes between replay/proving paths.

Recommendation

We recommend defining one canonical malformed-blob decoding policy (strict-fail or tolerant-fallback) and enforcing that same policy in both the Go and Rust implementations so equivalent L1 inputs always produce equivalent derivation outcomes. We also recommend adding differential tests that feed identical malformed payload corpora into both clients and assert the same accept/reject behavior and error surface.

Alleviation

[Mantle Network , 03/25/2026]: The issue has been fixed in this PR. <https://github.com/mantle-xyz/kona/pull/23>

[CertiK , 03/25/2026]: The team heeded the advice and resolved the issue in Kona PR [23](#) by aligning malformed-blob decoding behavior across Go and Rust

MAN-25 | Potential Divide By Zero During Base-Fee Verification

Category	Severity	Location	Status
Logical Issue	● Medium	consensus/misc/eip1559/eip1559.go (base-op-geth): 76~81, 93~101, 151~159	● Resolved

Description

In `eip1559.go`, the Arsia-era EIP-1559 parameter validation accepts `elasticity = 0` as valid whenever `denominator != 0`. `ValidateHolocene1559Params` rejects only `denominator == 0 && elasticity != 0`, but does not enforce `elasticity > 0`. The function `CalcBaseFee` later decodes these values from parent `extraData` and computes `parentGasTarget := parent.GasLimit / elasticity` without a guard, so a parent block carrying `elasticity = 0` can cause deterministic division-by-zero panic during child header verification. This runs on consensus verification paths, so the risk is chain liveness impact rather than local-only failure.

Recommendation

We recommend enforcing `elasticity > 0` in `MinBaseFee` `extraData` validation and reject zero at import boundaries. Moreover, we recommend adding validation in `CalcBaseFee` (or caller-side header validation) so malformed decoded parameters return deterministic errors instead of panicking.

Alleviation

[Mantle Network, 03/03/2026]: yes, a zero elasticity and zero denominator won't be rejected within `ValidateHolocene1559Params`. And it will be translated to local config value or default value. It handles the case where configurable 1559 params get enabled but not yet set in L1 SystemConfig contract. But that translation occurs if and only if `denominator = 0`. A zero elasticity but non-zero denominator will neither be rejected by validation nor translated to non-zero value.

Besides, SystemConfig contract revert all zero elasticity/denominator setting. So in my point of view, in practice, zero elasticity would not be in parent header.extraData.

Btw, Internal code review also informed the risk of division by 0. At that time, we double checked optimism's implementation and verified that they also accept and translate zero values in op-geth. Those values are given by op-node and apparently an advance translation within op-node would lead to simpler procedure in op-geth. we don't know why optimism decided to implement like that. As a follower of optimism, we think that advancing this translation without careful consideration might cause more problems so we choose the same path as optimism.

[CertiK, 03/03/2026]:

Based on dev team answer, we acknowledge that this condition is not expected to be reachable in production, as SystemConfig rejects zero elasticity/denominator updates; accordingly, such values should not be present in

parentHeader.extraData. We further acknowledge the dev team decision to preserve implementation parity with Optimism (op-node/op-geth) to mitigate compatibility risk from introducing unilateral translation behavior changes.

Given this context and rationale, the finding is considered resolved.

MAN-27 | Mantle RLP Decode Path Does Not Enforce Full Payload Consumption

Category	Severity	Location	Status
Coding Issue	● Medium	crates/protocol/derive/src/sources/mantle_blob.rs (base-kona): 219~226; op-node/rollup/derive/mantle_blob_source.go (base-mantle-v2): 193~206	● Resolved

Description

In the Mantle blob decode branch, the Rust implementation decodes an RLP `Vec<Bytes>` from a mutable slice and treats decode success as authoritative without checking that the slice was fully consumed. In theory, this permits payloads where a valid RLP list prefix is followed by trailing bytes that are silently ignored. However, in practice the derivation pipeline only processes blobs that have already passed lower-layer validation (EIP-4844 blob integrity, batcher address authentication, and L1 inclusion), and the privileged batcher constructs canonical RLP under normal operation. Triggering this ambiguity would therefore require a compromised or buggy batcher crafting a non-canonical payload. The practical likelihood is low, but the absence of a full-consumption check creates a latent cross-implementation parity risk between the Rust and Go Mantle blob sources and weakens defense-in-depth at the RLP parsing boundary.

Recommendation

As a hardening measure, we recommend enforcing canonical RLP parsing by rejecting Mantle-format decodes unless the input slice is fully consumed after `VecOfBytes::decode`. A differential regression test that feeds an RLP-list-plus-trailing-bytes payload and asserts consistent failure/handling behavior across Rust and Go Mantle blob sources would further strengthen confidence in cross-implementation parity.

Alleviation

[Mantle Network , 03/25/2026]: The issue has been fixed in this PR. <https://github.com/mantle-xyz/kona/pull/23>

[CertiK , 03/25/2026]: The team heeded the advice and resolved the issue in Kona PR [23](#) by enforcing full payload consumption for Mantle RLP decoding (rejecting trailing bytes).

MAN-30 | Unused DA Limits In Preconf Environment

Category	Severity	Location	Status
Logical Issue	● Medium	miner/preconf_checker.go (base-op-geth): 358~395, 463~511; miner/worker.go (base-op-geth): 566~631, 830~870	● Resolved

Description

The main block-building path enforces a per-block DA footprint budget via `daFootprintGasScalar` in `commitTransactions` and `commitFIFOTransactions`, rejecting transactions that would exceed the DA limit. The preconf execution path (`applyPreconfTransaction` / `applyTxWithResetEnv` in `preconf_checker.go`) has no equivalent DA footprint check and `daFootprintGasScalar` is never referenced anywhere in `preconf_checker.go`. This means the preconf system accepts transactions without any DA budget constraint.

When `fillTransactions` later replays preconfirmed transactions at seal time via `commitFIFOTransactions` (which does enforce the DA limit) transactions that exceed the DA footprint are rejected, breaking preconfirmation promises to users. Any external user can trigger this by submitting DA-heavy transactions during the preconf window. The `PreconfBufferBlock` mechanism (default 6 blocks) limits the blast radius but does not prevent the issue.

Recommendation

We recommend adding a DA footprint budget check in the preconf path or clarify the intended behavior.

Alleviation

[Mantle Network , 03/10/2026]: Issue acknowledged. Changes have been reflected in the commit hash: <https://github.com/mantlenetworkio/op-geth/commit/e8bf1d9ceac72e6cd8a47ebdbc5aac61846c3694>

[CertiK , 03/10/2026]: The team heeded the advice and resolved the issue by adding DA footprint checks and accounting to the preconfirmation path and propagating `daFootprintGasScalar` in the environment copy in commit [e8bf1d9ceac72e6cd8a47ebdbc5aac61846c3694](https://github.com/mantlenetworkio/op-geth/commit/e8bf1d9ceac72e6cd8a47ebdbc5aac61846c3694)

MAN-32 Token Ratio Scaling Can Underflow Optimism Gas Limit Pre-Arsia

Category	Severity	Location	Status
Incorrect Calculation	● Medium	crates/op-revm/src/handler.rs (base-revm): 126~163, 336~383	● Acknowledged

Description

In the pre-Arsia Optimism handler path, non-deposit transactions first have their intrinsic gas (`initial_gas` and `floor_gas`) validated by the mainnet handler, ensuring `initial_gas <= tx.gas_limit`. However, in `OpHandler::validate_initial_tx_gas` the Mantle-specific logic subsequently multiplies both `initial_gas` and `floor_gas` by `chain.token_ratio` without re-checking them against `tx.gas_limit`. This can make `initial_gas` exceed `tx.gas_limit` after scaling. Later, in `OpHandler::execution`, the handler computes `gas_limit = tx.gas_limit() - init_and_floor_gas.initial_gas` using plain `u64` subtraction, and then further adjusts `gas_limit` with `wrapping_sub` and `wrapping_div` when subtracting L1 cost and dividing by `token_ratio`. If the scaled `initial_gas` is greater than `tx.gas_limit`, this subtraction can underflow. As a result, the interpreter may either crash or execute with an oversized gas budget that is no longer bounded by the caller's original `tx.gas_limit`, breaking gas accounting and potentially allowing more work to be performed than was paid for.

Proof of Concept

This PoC demonstrates that the pre-Arsia Mantle gas path can accept a transaction whose intrinsic gas fits before scaling, then overflow the execution gas calculation after scaling by `token_ratio` causing panic.

Expected safe behavior:

- If `initial_gas` is increased by Mantle-specific `token_ratio` logic, the result should still be checked against `tx.gas_limit()`.
- Execution should never start with a gas budget derived from an underflowed subtraction.

Actual behavior:

- `validate_initial_tx_gas()` first calls the mainnet validator, which checks `initial_gas <= tx.gas_limit()`.
- The Mantle pre-Arsia path then multiplies `initial_gas` and `floor_gas` by `chain.token_ratio`.
- There is no second `initial_gas <= tx.gas_limit()` check after that scaling.
- `execution()` then computes `tx.gas_limit() - init_and_floor_gas.initial_gas` with plain `u64` subtraction.

If the scaled value is larger than `tx.gas_limit()`, that subtraction underflows.

Copy the following code in:

- `revm/crates/op-revm/src/handler.rs`

```
#[test]
fn test_pre_arsia_scaled_initial_gas_exceeds_tx_limit_and_panics_execution() {
    const TOKEN_RATIO: u64 = 3045;
    const TX_GAS_LIMIT: u64 = 30_000;

    let ctx = Context::op()
        .with_tx(
            OpTransaction::builder()
                .base(TxEnv::builder().gas_limit(TX_GAS_LIMIT))
                .enveloped_tx(Some(bytes!("01")))
                .build()
                .unwrap(),
        )
        .with_chain(L1BlockInfo {
            l2_block: Some(U256::ZERO),
            token_ratio: U256::from(TOKEN_RATIO),
            ..Default::default()
        })
        .modify_cfg_chained(|cfg| cfg.spec = OpSpecId::REGOLITH);

    let mut evm = ctx.build_op();
    let mut handler =
        OpHandler::<_, EVMEError<_, OpTransactionError>,
        EthFrame<EthInterpreter>>::new();

    let init = handler.validate_initial_tx_gas(&mut evm).unwrap();

    assert_eq!(init.initial_gas, 21_000 * TOKEN_RATIO);
    assert!(init.initial_gas > TX_GAS_LIMIT);

    let result = std::panic::catch_unwind(std::panic::AssertUnwindSafe(|| {
        let _ = handler.execution(&mut evm, &init);
    }));

    assert!(
        result.is_err(),
        "execution should panic when tx.gas_limit() - scaled initial_gas underflows"
    );
}
```

Expected Result

The result is a panic with message:

```
attempt to subtract with overflow
```

Recommendation

After scaling `initial_gas` and `floor_gas` by `token_ratio` in `OpHandler::validate_initial_tx_gas`, explicitly revalidate that the scaled `initial_gas` does not exceed `tx.gas_limit`, returning a deterministic invalid tx error if it does. In `OpHandler::execution`, replace the raw `tx.gas_limit() - init_and_floor_gas.initial_gas` with a checked subtraction and fail closed on underflow instead of relying on wrapping semantics. Avoid `wrapping_sub` / `wrapping_div` for gas limit derivations; use checked or saturating arithmetic and keep a single, validated gas budget that is consistent between normal and inspector execution paths.

Alleviation

[Mantle Network , 03/12/2026]:

In the pre-Arsia Optimism version, our `gas_limit` has already been multiplied by the token ratio. In practice, there is no situation where initial gas exceeds `gas_limit`, so we will not fix this issue for now since kona & revm not acts as sequencer but verifier, go-implementation would block the case that may leads to rust-implementation underflow.

[CertiK , 03/12/2026]:

Based on the team's explanation, we acknowledge the team's clarification that, in the pre-Arsia deployment, `op-revm` always receives a `gas_limit` already scaled by `tokenRatio`, so real L2 transactions satisfy `initial_gas * tokenRatio <= gas_limit` and the PoC's underflow path is not reachable in supported configurations. Additionally, the argument that the Go sequencer blocks the triggering condition is reasonable, but it means the Rust verifier's correctness depends on the Go implementation being correct and always running upstream. An independent verifier should handle all inputs soundly on its own. On this basis we consider the issue acknowledged, while still recommending that this invariant be documented and, where feasible, guarded by end-to-end integration tests to prevent regressions in future refactors.

MAN-33 | `decode_deposit()` Truncates `ethTxValue` And Creates Go/Rust Deposit Execution Divergence

Category	Severity	Location	Status
Logical Issue	● Medium	crates/protocol/derive/src/attributes/stateful.rs (base-kona): 97~100, 213~234; crates/protocol/protocol/src/deposits.rs (base-kona): 251~325; op-node/rollup/derive/deposit_log.go (base-mantle-v2): 35~98, 154~211	● Acknowledged

Description

In `unmarshal_deposit_version1`, the Rust decoder parses `eth_tx_value` with `decode_u128_field()`, which reads only the low 16 bytes of the 32-byte ABI slot and never rejects non-zero high bytes. The Go `op-node` deposit decoder, by contrast, parses the same version-1 field as a full `uint256` in `unmarshalDepositVersion1()`. As a result, the same L1 deposit log can be decoded into different deposit transactions in Rust and Go whenever `ethTxValue` has any non-zero high 128 bits.

This happens during normal derivation as `derive_deposits()` feeds deposit-contract logs directly into `decode_deposit()` during payload construction, while the Go node derives the same deposit logs through `UnmarshalDepositLogEvent()` and `unmarshalDepositVersion1()`. Therefore, a version-1 deposit with `ethTxValue >= 2^128` causes the Rust path to truncate the transfer amount while the Go path preserves the full value, creating a real Go/Rust deposit-derivation mismatch from the same L1 input.

Proof of Concept

This PoC demonstrates that the same version-1 Mantle deposit log is decoded differently by Go and Rust.

For the Rust part:

Add this file at:

- `kona/crates/protocol/protocol/tests/deposit_divergence_poc.rs`

For the Go Part

Add this file at:

- `op-geth/preconf/deposit_log_poc_test.go`

```
package preconf

import (
    "encoding/binary"
    "math/big"
    "testing"

    "github.com/ethereum/go-ethereum/common"
    "github.com/ethereum/go-ethereum/core/types"
)

func TestUnmarshalDepositVersion1PreservesHighBitsPoC(t *testing.T) {
    from :=
common.HexToHash("0x0000000000000000000000000000000000000000000000000000000000000000")
    to :=
common.HexToHash("0x0000000000000000000000000000000000000000000000000000000000000000")

    data := make([]byte, 224)

    // ABI dynamic bytes header
    data[31] = 32
    data[63] = 137

    // msg.value / ethValue = 2 ether
    big.NewInt(2_000_000_000_000_000_000).FillBytes(data[128:160])

    // _ethTxValue = 2^128 + 1 ether
    hugeEthTxValue, ok := new(big.Int).SetString(
        "0000000000000000000000000000000000000000000000000000000000000000de0b6b3a7640000",
        16,
    )
    if !ok {
        t.Fatal("failed to build huge eth tx value")
    }
    hugeEthTxValue.FillBytes(data[160:192])

    // gas limit = 21000
    binary.BigEndian.PutUint64(data[192:200], 21_000)

    log := &types.Log{
        Topics: []common.Hash{
            DepositEventABIHash,
            from,
            to,
            DepositEventVersion1,
        },
        Data: data,
```

```

    }

    dep, err := UnmarshalDepositLogEvent(log)
    if err != nil {
        t.Fatalf("unexpected decode error: %v", err)
    }

    if dep.EthValue == nil ||
dep.EthValue.Cmp(big.NewInt(2_000_000_000_000_000_000)) != 0 {
        t.Fatalf("expected eth value 2 ether, got %v", dep.EthValue)
    }

    if dep.EthTxValue == nil || dep.EthTxValue.Cmp(hugeEthTxValue) != 0 {
        t.Fatalf("expected full eth tx value %v, got %v", hugeEthTxValue,
dep.EthTxValue)
    }
}

```

Expected Result:

For the same deposit log:

- Rust decodes:
 - `eth_value = 2000000000000000000`
 - `eth_tx_value = 10000000000000000000`
- Go decodes:
 - `EthValue = 2000000000000000000`
 - `EthTxValue = 0x1000000000000000000de0b6b3a7640000`

Recommendation

We recommend rejecting any version-1 deposit whose `mint`, `eth_value`, or `eth_tx_value` field does not fit in the Rust-side representation before constructing the deposit transaction. If the protocol intends to support full `uint256` deposit values, widen the Rust deposit transaction and revm abstractions so they preserve the same value domain as Go.

Alleviation

[Mantle Network , 03/13/2026]:

Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement. The rational is that a real tx with eth value > `u128.max` is not possible. Even if it happens, there will be no fund lost but a halt of zk proof produce.

[Certik , 03/13/2026]:

We acknowledge the team's plan to address the Go/Rust `ethTxValue` decoding divergence after this audit engagement.

MAN-35 | Duplicate `getBlobsV2` Hit Metric Increment

Category	Severity	Location	Status
Coding Issue	● Medium	eth/catalyst/api.go (base-op-geth): 575-617	● Resolved

Description

The `getBlobsV2` implementation increments `getBlobsV2RequestHit` before fetching blob sidecars and again after verifying that all returned blob entries are non nil. Counting a hit both before and after validation inflates successful hit telemetry and makes miss metrics unreliable for monitoring and alerting.

Recommendation

We recommend incrementing `getBlobsV2RequestHit` exactly once, only after all requested blobs have been successfully fetched and validated as non-nil, and to keep early-return paths counted only as misses.

Alleviation

[Mantle Network , 03/10/2026]: Issue acknowledged. Changes have been reflected in the commit hash:

<https://github.com/ethereum-optimism/op-geth/commit/612cba57425dca13cd32da53612fe0d39f4207c1>

[CertiK , 03/10/2026]: The team heeded the advice and resolved the issue by removing the second increment in commit [612cba57425dca13cd32da53612fe0d39f4207c1](https://github.com/ethereum-optimism/op-geth/commit/612cba57425dca13cd32da53612fe0d39f4207c1)

MAN-36 | Inconsistency Between Post-Arsia Decoding Behavior Between Kona And Op-Node.

Category	Severity	Location	Status
Logical Issue	● Medium	crates/protocol/derive/src/sources/mantle_blob.rs (base-kona): 219~235; crates/protocol/derive/src/sources/mantle_ethereum.rs (base-kona): 77~83	● Resolved

Description

After Arsia activation, the batcher permanently switches from Mantle RLP blob format to standard OP blob format. Go's `DataSourceFactory.OpenData` detects this transition and stops routing to `MantleBlobDataSource`, switching to `BlobDataSource` for all subsequent blocks. Rust's `MantleEthereumDataSource.next()` has no equivalent routing: it unconditionally delegates to `MantleBlobSource`, which attempts Mantle RLP decode on every block regardless of fork state.

The divergence is normally silent, since Mantle RLP decode fails and `MantleBlobSource` falls back to standard per-blob decoding, producing the same output as Go. The risk materialises if a standard OP blob's decoded payload happens to be a structurally valid RLP list of byte arrays: `MantleBlobSource` would accept it as Mantle-format and return different frames than Go's `BlobDataSource`, causing a consensus divergence between the kona prover and the op-node.

Recommendation

We recommend implementing a timestamp-based fork routing in `MantleEthereumDataSource.next()` using the already-stored `ecotone_timestamp` field (equal to `MantleArsiaTime`) and routing to standard `BlobSource` when `block_time >= ecotone_timestamp`, to `MantleBlobSource` when `block_time >= mantle_everest_timestamp`, and to `CallDataSource` otherwise. Alternatively, we recommend clarifying the intended behavior

Alleviation

[Mantle Network , 03/25/2026]: The issue has been fixed in this PR. <https://github.com/mantle-xyz/kona/pull/23>

[CertiK , 03/25/2026]: The team heeded the advice and addressed MAN-36 in Kona PR [23](#) by improving Rust/Go parity for post-Arsia blob decoding via the updated Mantle blob decoding behavior.

MAN-37 Failed OP Deposit Mints BVM_ETH After Commit, Leaving Journal And Warm-Cache Dirty For Next Tx

Category	Severity	Location	Status
Coding Issue	● Medium	crates/op-revm/src/handler.rs (base-revm): 531~594; crates/op-revm/src/transaction/bvm_eth.rs (base-revm): 84~124	● Resolved

Description

In `OpHandler::catch_error`, when a deposit transaction fails with a tx error, the handler reverts the journal to the default checkpoint, applies the "mint always persists" rule and calls `journal.commit_tx()`. It then calls `BvmEth::process_eth_deposit(evm.ctx(), true)` to mint BVM_ETH. All BVM_ETH operations are performed after `commit_tx()` and are never followed by a second commit or by `discard_tx()`. The shared journal therefore retains uncommitted BVM_ETH state, logs, and warmed address entries. Cleanup only runs `clear_tx_l1_cost()`, `local_mut().clear()`, and `frame_stack().clear()`; the journal transaction boundary is not finalized. For non deposit errors, the handler returns `Err(error)` without calling `journal.discard_tx()`, unlike the base handler, so partial journal state from the failed tx can leak to the next transaction.

Proof of Concept

This PoC demonstrates that `OpHandler::catch_error()` leaves the shared transaction journal in an unfinalized state across error paths.

It validates two cases:

- A failed deposit writes BVM_ETH state and logs after `journal.commit_tx()`, and those stale logs are returned by the next successful transaction executed on the same EVM context.
- A non-deposit error returns without `journal.discard_tx()`, leaving prior journal entries live and the transaction boundary unadvanced.

To re-run the PoC copy the following test code in:

- `revm/crates/op-revm/src/handler.rs`

Test 1: Failed Deposit Log Leaks Into Next Transaction

```

#[test]
fn test_failed_deposit_bvm_eth_log_leaks_into_next_tx() {
    let caller = Address::from([0x11; 20]);
    let next_caller = Address::from([0x22; 20]);
    let mint_amount = 100u64;

    let ctx = Context::op()
        .modify_tx_chained(|tx| {
            tx.base.caller = caller;
            tx.deposit.source_hash = B256::from([1u8; 32]);
            tx.deposit.mint = Some(mint_amount.into());
            tx.deposit.eth_value = Some(mint_amount.into());
        })
        .modify_cfg_chained(|cfg| cfg.spec = OpSpecId::REGOLITH);

    let mut evm = ctx.build_op();
    let mut handler =
        OpHandler::<_, EVMError<_, OpTransactionError>,
        EthFrame<EthInterpreter>>::new();

    let result = handler
        .catch_error(
            &mut evm,
            EVMError::Transaction(OpTransactionError::HaltedDepositPostRegolith),
        )
        .unwrap();

    match result {
        ExecutionResult::Halt { reason, .. } => {
            assert_eq!(reason, OpHaltReason::FailedDeposit);
        }
        _ => panic!("Expected Halt result"),
    }

    assert_eq!(evm.ctx().journal().inner.transaction_id, 1);
    assert_eq!(evm.ctx().journal().inner.logs.len(), 1);
    assert!(!evm.ctx().journal().inner.journal.is_empty());

    evm.ctx().modify_tx(|tx| {
        *tx = OpTransaction::builder()
            .base(TxEnv::builder().gas_limit(21_000).caller(next_caller))
            .enveloped_tx(Some(bytes!("01")))
            .build()
            .unwrap();
    });

    let frame_result = FrameResult::Call(CallOutcome::new(
        InterpreterResult {
            result: InstructionResult::Stop,

```

```
        output: Bytes::new(),
        gas: Gas::new(21_000),
    },
    0..0,
));

let next_result = handler.execution_result(&mut evm, frame_result).unwrap();

match next_result {
    ExecutionResult::Success { logs, .. } => {
        assert_eq!(logs.len(), 1);
        assert_eq!(logs[0].address, BvmEth::ADDRESS);
        assert_eq!(logs[0].data.topics()[0], BvmEth::MINT_SELECTOR);
    }
    _ => panic!("Expected Success result"),
}
```

Test 2: Non-Deposit Error Leaves Journal Unfinalized

```
#[test]
fn test_non_deposit_error_leaves_journal_unfinalized() {
    let caller = Address::from([0x33; 20]);

    let mut evm = Context::op()
        .modify_tx_chained(|tx| tx.base.caller = caller)
        .build_op();
    let handler =
        OpHandler::<_, EVMError<_, OpTransactionError>,
        EthFrame<EthInterpreter>>::new();

    let mut caller_account =
        evm.ctx().journal_mut().load_account_mut(caller).unwrap();
    caller_account.incr_balance(U256::from(1));
    drop(caller_account);

    assert!(!evm.ctx().journal().inner.journal.is_empty());
    assert_eq!(evm.ctx().journal().inner.transaction_id, 0);

    let err = handler
        .catch_error(&mut evm, EVMError::Custom("boom".into()))
        .unwrap_err();

    match err {
        EVMError::Custom(message) => assert_eq!(message, "boom"),
        _ => panic!("Expected custom error"),
    }

    assert!(!evm.ctx().journal().inner.journal.is_empty());
    assert_eq!(evm.ctx().journal().inner.transaction_id, 0);
}
```

Passing both tests means:

For Test 1:

- The failed deposit path leaves a BVM_ETH mint log in `journal.inner.logs`.
- The next successful transaction receives that stale mint log in its own `ExecutionResult::Success.logs`.
- Therefore, a failed transaction can contaminate the next transaction's receipt/output stream.

For Test2:

- A non-deposit error can return without clearing the active journal.
- `transaction_id` is not advanced.
- The OP override does not mirror the base handler's `discard_tx()` behavior.

Expected Result

For the failed-deposit case:

- After `catch_error()`, the journal still contains one log and non-empty journal entries.
- The next successful transaction returns the stale BVM_ETH mint log in its `logs` field.

For the non-deposit case:

- `catch_error()` returns the error unchanged.
- The journal remains non-empty.
- `transaction_id` stays at `0`, proving no discard/finalize occurred.

Recommendation

Align `OpHandler::catch_error` with the base handler so every error path finalizes the journal before returning. For non deposit errors, call `evm.ctx().journal_mut().discard_tx()` before returning `Err(error)` and clear warmed addresses and pending logs. For failed deposits, perform BVM_ETH mint either in the same transaction scope as the nonce bump and balance increment and then call a single `commit_tx()`, or perform it in an isolated journal scope, commit that scope and clear per tx caches so the next tx starts with a clean journal.

Alleviation

[Mantle Network , 03/24/2026]: The issue has been fixed in this PR. <https://github.com/mantle-xyz/revm/pull/25>

[CertiK , 03/25/2026]: The team heeded the advice and resolved the issue in revm PR [25](#) by fixing the failed-deposit `OpHandler::catch_error` journal/cleanup flow to prevent BVM_ETH/log leakage into subsequent transactions.

MAN-38 | Early Ecotone/Isthmus Activation In Rust Creates Go/Rust Divergence

Category	Severity	Location	Status
Logical Issue	● Medium	crates/node/engine/src/versions.rs (base-kona): 65~69; crates/proof/executor/src/builder/assemble.rs (base-kona): 51~52, 88; crates/protocol/derive/src/attributes/stateful.rs (base-kona): 175~178; crates/protocol/genesis/src/rollup.rs (base-kona): 344~351, 419~425; crates/protocol/hardforks/src/utis.rs (base-kona): 82~90; crates/protocol/protocol/src/batch/single.rs (base-kona): 178~182; op-node/rollup/mantle_types.go (base-mantle-v2): 167~176	● Resolved

Description

In `rollup.rs`, `is_ecotone_active()` short-circuits to `true` when `is_mantle_skadi_active(timestamp)` holds, and `is_isthmus_active()` does the same. In Go, the op-node function `AlignOpWithMantle()` sets `EcotoneTime = MantleArsiaTime` and `IsthmusTime = MantleArsiaTime`, and the Go tests explicitly assert that both forks are inactive before Arsia. Because `MantleSkadi < MantleLimb < MantleArsia` in the fork ordering, there is an epoch `[mantle_skadi_time, mantle_arsia_time)` where Rust considers Ecotone and Isthmus active but Go does not.

The Rust predicates are used to gate derivation behavior, batch validation, proof-side header construction, and engine version selection. For blocks in `[Skadi, Arsia)`, kona therefore applies Ecotone/Isthmus-era rules while the Go `op-node` still treats those forks as inactive, creating a real Go/Rust derivation and proof-execution mismatch.

Recommendation

We recommend changing `is_ecotone_active()` and `is_isthmus_active()` for Mantle chains to gate on `is_mantle_arsia_active()` (matching Go's `EcotoneTime = ArsiaTime`), or explicitly document and test the intended fork mapping if Skadi is the intentional Ecotone activation point and verify that this divergence from Go is acceptable.

Alleviation

[Mantle Network, 03/24/2026]: Issue acknowledged. We understand this is fixed in Kona PR #23.

[CertiK, 03/25/2026]: The team heeded the advice and resolved the issue in Kona PR 23 by aligning Rust fork activation behavior with Go's Arsia-based timing.

MAN-03 | KMS Signer Client Created But Never Closed

Category	Severity	Location	Status
Volatile Code	Minor	op-service/crypto/signature.go (base-mantle-v2): 82, 89; op-service/txmgr/txmgr.go (base-mantle-v2): 206~209	Resolved

Description

The `HSM` / `KMS` signing path creates a long-lived Google `KMS` client, but no explicit shutdown path closes it. In `txmgr` flows this client is wrapped into a signer closure and the handle is not returned to a component that owns lifecycle cleanup. This can lead to resource leakage (open gRPC channels/background resources) across long-running processes, repeated initialization paths, or tests. While process exit eventually reclaims resources, graceful shutdown does not currently close this client explicitly.

Recommendation

We recommend adding explicit lifecycle ownership and closure for KMS clients:

- Make ownership explicit by having the signer setup return a cleanup function alongside the signer function/factory.
- Register that cleanup in the component shutdown path (e.g., tx manager/service Close/Stop).
- Ensure init error paths call cleanup if the signer client was already created.

Alleviation

[Mantle Network , 02/24/2026]: Issue acknowledged. Changes have been reflected in the commit hash:

<https://github.com/mantlenetworkio/mantle-v2/commit/d57ef11baa0a9f065555fc0bc1ab1f3209b10fba>

[CertiK , 02/25/2026]: The team heeded the advice and resolved the issue by introducing explicit signer lifecycle ownership for the HSM/KMS path in commit [d57ef11baa0a9f065555fc0bc1ab1f3209b10fba](https://github.com/mantlenetworkio/mantle-v2/commit/d57ef11baa0a9f065555fc0bc1ab1f3209b10fba) and [b12ee72f84ab3b10c4220614ab79cb6c225e0dc7](https://github.com/mantlenetworkio/mantle-v2/commit/b12ee72f84ab3b10c4220614ab79cb6c225e0dc7).

MAN-05 | Missing Explicit Low-S Canonicalization In KMS Signature Conversion

Category	Severity	Location	Status
Logical Issue	● Minor	op-service/hsm/hsm_signer.go (base-mantle-v2): 93~107	● Resolved

Description

The HSM signing flow in SignHash converts DER-encoded ECDSA output (r, s) from Google Cloud KMS into Ethereum transaction signature format [R || S || V] after recovery-ID probing. While the implementation verifies signature component sizes and recovered address correctness, it does not explicitly enforce low-s canonicalization before returning the final signature bytes. Ethereum transaction validation expects canonical ECDSA signatures, and high-s signatures can introduce malleability-related incompatibilities across clients, relayers, or strict validation paths. In this implementation, the issue is mainly a robustness concern rather than a direct key-compromise vector, and could manifest as intermittent transaction submission failures or retry loops if non-canonical signatures are ever produced by the upstream signer.

Recommendation

Add an explicit low-s normalization/check step during signature conversion in `SignHash`, and reject non-canonical outputs if normalization is not applied. This ensures deterministic, provider-independent behavior and prevents transaction-liveness issues caused by signature canonicalization edge cases.

Alleviation

[Mantle Network , 03/03/2026]: Issue acknowledged. Changes have been reflected in the commit hash:

<https://github.com/mantlenetworkio/mantle-v2/commit/36ace299ecbb4edf978fbc29b31be2aae3b170a8>

[Certik , 03/03/2026]: The team heeded the advice and resolved the issue by applying an explicit low-s normalization step in commit [36ace299ecbb4edf978fbc29b31be2aae3b170a8](https://github.com/mantlenetworkio/mantle-v2/commit/36ace299ecbb4edf978fbc29b31be2aae3b170a8)

MAN-07 | `trustrpc` Backup-Sync Flag Type Mismatch

Category	Severity	Location	Status
Inconsistency	● Minor	op-node/config/backup_sync_rpc.go (base-mantle-v2): 43, 60; op-node/flags/flags.go (base-mantle-v2): 41	● Resolved

Description

The backup-sync trust flag is declared as a **string** flag but consumed as a **boolean** value.

- In `flags.go`, `BackupL2UnsafeSyncRPCTrustRPC` is defined as `*cli.StringFlag`.
- In `mantle_service.go`, the same flag is read using `ctx.Bool(...)` and assigned to `TrustRPC`.
- The resulting `TrustRPC` value is passed into backup sync config and directly affects RPC verification behavior.

This mismatch can cause operator confusion, inconsistent CLI behavior, and incorrect assumptions about whether backup RPC responses are trusted/untrusted.

Recommendation

We recommend using a consistent flag type and access pattern for the `BackupL2UnsafeSyncRPCTrustRPC` flag.

Alleviation

[Mantle Network , 02/24/2026]: Issue acknowledged. Changes have been reflected in the commit hash:

<https://github.com/mantlenetworkio/mantle-v2/commit/fabcbac696e75efada14ce09850487b548e79910>

[CertiK , 02/25/2026]: The team heeded the advice and resolved the issue by using a consistent flag type for

``BackupL2UnsafeSyncRPCTrustRPC`` in commit [fabcbac696e75efada14ce09850487b548e79910](https://github.com/mantlenetworkio/mantle-v2/commit/fabcbac696e75efada14ce09850487b548e79910)

MAN-17 | Gas Limit Setter Enforces Minimum But No Explicit Maximum Bound

Category	Severity	Location	Status
Coding Issue, Inconsistency	● Minor	packages/contracts-bedrock/src/L1/SystemConfig.sol (base-mantle-v2): 216~218, 280~286	● Resolved

Description

`SystemConfig.setGasLimit(uint64)` currently enforces only `minimumGasLimit()` and does not enforce an explicit maximum cap. This is a meaningful divergence from upstream OP Stack behavior, where gas-limit updates are bounded by both minimum and maximum invariants. As a result, governance can set values that are valid on-chain but outside practical proving and execution envelopes, creating avoidable liveness/performance risk during operations and incident response.

Recommendation

We recommend restoring upstream-equivalent bounded validation by enforcing an explicit `maximumGasLimit()` check in `setGasLimit`. We also recommend calibrating the max bound against measured proving/execution capacity, and documenting an approved safe operating range in runbooks so emergency governance actions cannot exceed validated limits. Alternatively, if this is a design decision we recommend clarifying the rational behind it.

Alleviation

[Mantle Network , 02/24/2026]: Issue acknowledged. Changes have been reflected in the commit hash: <https://github.com/mantlenetworkio/mantle-v2/commit/e33a4f89741c180d133473cf44a0e76259ed4d3c>

[CertiK , 02/25/2026]: The team heeded the advice and addressed the issue in commit [e33a4f89741c180d133473cf44a0e76259ed4d3c](https://github.com/mantlenetworkio/mantle-v2/commit/e33a4f89741c180d133473cf44a0e76259ed4d3c)

MAN-19 | Potential Nil-Dereference Crash Paths

Category	Severity	Location	Status
Coding Style	Minor	op-node/cmd/genesis/cmd.go (base-mantle-v2): 172; op-node/node/node.go (base-mantle-v2): 493~499; op-service/sources/backup_sync_client.go (base-mantle-v2): 61, 73, 185	Resolved

Description

At the pointed locations, there are potential missing nil-checks that could cause panic. In particular:

- In `node.go`, the function `initL1BeaconAPI` enters the beacon-required branch when either `EcotoneTime` or `MantleEverestTime` is scheduled. If `cfg.Beacon == nil`, the returned error string always dereferences `*cfg.Rollup.EcotoneTime`. When only `MantleEverestTime` is set and `EcotoneTime` is nil, this path panics while formatting the error message.
- In `cmd.go` the Mantle-specific `op-node genesis l2` flow, L1 start-block resolution dereferences `config.L1StartingBlockTag` and later logs `l1StartBlock.Hash()` before the config sanity check runs. If `L1StartingBlockTag` is absent, or if neither block hash nor block number is set, the command can panic with a nil-pointer dereference instead of returning a controlled validation error. This is primarily an availability and operator-experience issue in CLI/deployment workflows, but it still weakens fail-safe behavior and can be triggered by malformed or partially migrated config inputs.
- In `backup_sync_client.go`, the function `NewSyncClient(...)` stores `receiver receivePayload` directly and `fetchUnsafeBlockFromRpc(...)` later invokes `s.receivePayload(...)` without a nil check. In the current repository wiring, this callback always appears non-nil (constructed from `BlockReceiver.OnUnsafeL2Payload`). However, because `NewSyncClient` is exported, future integrations or refactors could pass a nil callback and trigger a runtime panic when payload forwarding executes.

Recommendation

We recommend ensuring that all fields are checked to be non-nil before accessing its functions/state to avoid panic.

Alleviation

[Mantle Network, 02/25/2026]: Issue acknowledged. Changes have been reflected in the commit hash:

<https://github.com/mantlenetworkio/mantle-v2/commit/1c26a80c37fd8cad84abc29e08bee68d5e3bfa8a>

[Certik, 02/25/2026]: The team heeded the advice and resolved the issue by adding the suggested nil checks in commit

[\[1c26a80c37fd8cad84abc29e08bee68d5e3bfa8a\]\(https://github.com/mantlenetworkio/mantle-v2/commit/1c26a80c37fd8cad84abc29e08bee68d5e3bfa8a\)](https://github.com/mantlenetworkio/mantle-v2/commit/1c26a80c37fd8cad84abc29e08bee68d5e3bfa8a).

MAN-23 | `eth_estimateTotalFee` Does Not Preserve `blockHash` Selector Semantics

Category	Severity	Location	Status
Coding Issue, Volatile Code	● Minor	internal/ethapi/api.go (base-op-geth): 1111~1131	● Resolved

Description

`EstimateTotalFee` accepts `blockHash` selectors but unconditionally converts the selector to a block number before calling `DoEstimateGas`, which then performs state and header resolution by number. As a result, hash-pinning and `requireCanonical` expectations are not preserved end-to-end: although the caller provides a block hash, estimation is executed by block number rather than by the caller-provided hash. The standard `EstimateGas` endpoint does not exhibit this behavior and passes the caller's selector through unchanged. While most current callers use `latest` (where the existing behavior is acceptable), silently accepting and reinterpreting `blockHash` input creates a reliability footgun for integrators and internal tooling that treat `BlockNumberOrHash` uniformly.

Recommendation

If hash-targeted mode is not intended for this endpoint, reject `blockHash` inputs explicitly with a descriptive error. Otherwise, preserve the caller's original selector throughout the estimation flow, consistent with how `EstimateGas` handles the same parameter. As a stronger hardening step, resolve state and header once and pass the resolved context into gas estimation to remove the redundant second lookup.

Alleviation

[Mantle Network , 03/04/2026]: Issue acknowledged. Changes have been reflected in the commit hash:

<https://github.com/mantlenetworkio/op-geth/commit/eba5278110c48657d53706c1e04681a2abd1a7f7>

[CertiK , 03/09/2026]: The team heeded the advice and resolved the issue by preserving hash-based block selectors through to `DoEstimateGas` instead of unconditionally converting them to block numbers, in commit [eba5278110c48657d53706c1e04681a2abd1a7f7](https://github.com/mantlenetworkio/op-geth/commit/eba5278110c48657d53706c1e04681a2abd1a7f7)

MAN-24 | Unchecked `uint256` `tokenRatio` Is Narrowed To Fixed-Width Gas Arithmetic, Producing Wrap-Vs-Fail-Stop Divergence

Category	Severity	Location	Status
Inconsistency, Incorrect Calculation	Minor	crates/op-revm/src/handler.rs (base-revm): 148~159, 379~421, 428~439; packages/contracts-bedrock/src/L2/GasPriceOracle.sol (base-mantle-v2): 119~124; core/state_transition.go (base-op-eth): 703, 727~763, 848~852; core/txpool/validation.go (base-op-eth): 285, 307~323	Resolved

Description

`GasPriceOracle.setTokenRatio(uint256)` accepts any `uint256` input without a protocol-level upper-bound validation, and that value is later consumed by execution clients as a fixed-width gas scalar. In `op-eth`, the value is read as `Uint64` (`Big().Uint64()`) and used in pre-Arsia gas accounting (`gas = gas * tokenRatio`, `floorDataGas * tokenRatio`, `gasRemaining / tokenRatio`, and refund scaling), so oversized values are truncated modulo `2^64` before validation and can silently alter intrinsic-gas checks and accounting outcomes. In `op-revm`, the same chain value is converted via `try_into / unwrap / assert` before arithmetic, so out-of-range values can fail conversion or trigger assertions and abort execution paths (`panic / Err::from`), effectively turning a valid on-chain ratio update into a hard failure mode. The net effect is a protocol-level parity and liveness divergence: one client can continue with wrapped arithmetic while another aborts, causing inconsistent block/fee behavior across implementations.

Recommendation

We recommend enforcing a protocol-wide bound on `tokenRatio` before narrowing to fixed-width types, validate against consensus-safe maxima at the first consumer boundary, and align checked handling across implementations so both clients accept and reject identically. Add parity tests at integer boundary values, especially near `2^64` and pre-Arsia intrinsic-gas cases, to prove consistent behavior.

Alleviation

[Mantle Network, 03/05/2026]: Issue acknowledged. Changes have been reflected in the commit hash:

<https://github.com/mantlenetworkio/mantle-v2/commit/3871bbadb6d6f6b812d28c8045ebdec20bf1f06>

<https://github.com/mantle-xyz/revm/pull/24>

[CertiK, 03/09/2026]: The team heeded the advice and resolved the issue by bounding `setTokenRatio` to `[1, 2^64-1]` at the contract level in commit [3871bba](#)

MAN-29 | Potential Nil Pointer Dereference In `makeEnv`

Category	Severity	Location	Status
Logical Issue	Minor	miner/worker.go (base-op-gets): 395~404	Resolved

Description

In `makeEnv`, when `StateAt(parent.Root)` fails on an Optimism-configured chain, the code falls back to `historicalBackend.StateAtBlock()`. However, `state.Copy()` is called immediately after `StateAtBlock` without checking whether it returned a non-nil error. If `StateAtBlock` fails, `state` is nil and `state.Copy()` panics with a nil-pointer dereference. Additionally, `parentBlock` from `GetBlockByHash` is not validated for nil before being passed to `StateAtBlock`.

Recommendation

We recommend adding nil-checks for `state` and `parentBlock` before using them.

Alleviation

[Mantle Network , 03/10/2026]: Issue acknowledged. Changes have been reflected in the commit hash:
<https://github.com/mantlenetworkio/op-gets/commit/10b003ce79e29e37c77cc08e1c13ff0b3ffa0793>

[CertiK , 03/10/2026]: The team heeded the advice and resolved the issue by adding nil checks in commit
[10b003ce79e29e37c77cc08e1c13ff0b3ffa0793](https://github.com/mantlenetworkio/op-gets/commit/10b003ce79e29e37c77cc08e1c13ff0b3ffa0793)

MAN-34 | Potential Panic Of CalcBaseFee On Missing BlobGasUsed

Category	Severity	Location	Status
Coding Issue	Minor	consensus/misc/eip1559/eip1559.go (base-op-geth): 76~88	Resolved

Description

The base-fee calculation path panics when a parent header on the Arsia (Jovian) path has a nil `BlobGasUsed` field.

```
if config.IsMantleArsia(parent.Time) {
    if parent.BlobGasUsed == nil {
        panic("Jovian parent block has nil BlobGasUsed")
    } else if *parent.BlobGasUsed > parent.GasUsed {
        parentGasMetered = *parent.BlobGasUsed
    }
}
```

A missing `BlobGasUsed` on a parent header causes a process panic and crash instead of a handled error. A malformed or partially populated parent header (or RPC/import race) can therefore bring down the node, causing availability and operational disruption.

Recommendation

Replace the `panic` with a handled error: return a descriptive error from `CalcBaseFee` (or validate earlier) so callers can reject the header and record metrics instead of crashing.

Alleviation

[Mantle Network , 03/10/2026]: IsMantleArsia implies IsCancun in the OP-Stack fork sequence, so BlobGasUsed is guaranteed to be initialized for any Arsia block and it cannot be nil. The panic is an intentional invariant assertion: if data integrity is somehow violated, crashing is safer than silently propagating a corrupted base fee. Additionally, changing CalcBaseFee to return an error would require signature changes across 17+ call sites with no practical benefit. We consider the current behavior correct.

[CertiK , 03/10/2026]: Based on the dev team's answer, we acknowledge that this condition is not expected to be reachable in production, as the SystemConfig contract rejects zero elasticity/denominator updates; accordingly, such values should not be present in parentHeader.extraData. We further acknowledge the dev team's decision to preserve implementation parity with Optimism (op-node/op-geth) to avoid introducing compatibility risks through unilateral translation behavior changes.

Given this context and rationale, the finding is considered resolved.

MAN-39 | Unbounded `tokenRatio` Configuration Can Trigger Runtime Panics In OP-REVM Fee Logic

Category	Severity	Location	Status
Coding Style	Minor	crates/handler/src/handler.rs (base-revm): 151~159, 416~421; packages/contracts-bedrock/src/L2/GasPriceOracle.sol (base-mantle-v2): 119~123	Resolved

Description

The L2 `GasPriceOracle.setTokenRatio(uint256)` setter accepts an arbitrary 256-bit value and stores it without upper-bounds validation. The OP-REVM consumer performs fallible conversions and uses `unwrap/assert` which panic on out-of-range values, e.g.:

```
// packages/contracts-bedrock/src/L2/GasPriceOracle.sol
function setTokenRatio(uint256 _tokenRatio) external onlyOperator {
    uint256 previousTokenRatio = tokenRatio;
    tokenRatio = _tokenRatio;
    emit TokenRatioUpdated(previousTokenRatio, tokenRatio);
}
```

```
// revm handler (relevant path)
let token_ratio_u64: u64 = chain.token_ratio.try_into().unwrap();
assert!(token_ratio_u64 <= i64::MAX as u64, "token_ratio {token_ratio_u64} exceeds i64::MAX");
initial_gas.initial_gas = initial_gas.initial_gas
    .checked_mul(token_ratio_u64)
    .ok_or(InvalidTransaction::CallerGasLimitMoreThanBlock)?;
```

If `tokenRatio` is set to an extreme value (misconfiguration, migration error, or malicious privileged action) this path can panic and crash processing, causing availability/DoS and potential replay/proving issues.

Recommendation

We recommend validating and bounding `tokenRatio` at the source, and replacing all Rust `unwrap/assert` conversions with checked error-returning paths so out-of-range values fail closed instead of panicking. Moreover, add minimal telemetry and boundary regression tests for representative valid and invalid values.

Alleviation

[Mantle Network , 03/10/2026]: Is this a similar issue with MAN24 that is resolved by limiting token ratio to $[1, 2^{64}-1]$ in contract level?

[CertiK , 03/10/2026]: This issue is effectively addressed by the same mitigation as MAN-24, which bounds `tokenRatio` at the contract level and updates the OP-REVM consumer accordingly; as a result, the unbounded-configuration panic path described here is no longer reachable. The finding is therefore considered resolved.

MAN-40 | Panic DoS When RPC Code Bytes Start With EIP-7702 Magic But Are Not Valid EIP-7702

Category	Severity	Location	Status
Logical Issue	Minor	crates/bytecode/src/bytecode.rs (base-revm): 79~83, 95~103; crates/bytecode/src/eip7702.rs (base-revm): 34~41; crates/database/src/alloydb.rs (base-revm): 81~84, 89~96	Resolved

Description

When account code is loaded from an RPC provider via AlloyDB, the database and bytecode layer assumes it is well-formed and turns decode failures into process panics instead of recoverable errors. Code whose first two bytes equal

`EIP7702_MAGIC_BYTES` is treated as EIP-7702 and parsed with `Eip7702Bytecode::new_raw(bytes)?`, valid EIP-7702 designators are exactly 23 bytes so any other length makes that call return `Err`. In `AlloyDB::basic_async_ref`, the bytes from the provider are passed to `Bytecode::new_raw(...)`, which uses `.expect("Expect correct bytecode")` on the result of `new_raw_checked`, so any decode error aborts the process. Separately, `AlloyDB::block_hash_async_ref` calls `get_block_by_number` and then `block.unwrap()`. If the provider returns `None`, the unwrap panics.

Recommendation

We recommend using fallible bytecode construction in RPC-ingestion paths via `Bytecode::new_raw_checked()` and propagating decode failures as recoverable provider/database errors instead of panicking. We also recommend gating EIP-7702 parsing on the active fork and replacing `block.unwrap()` in `BLOCKHASH` handling with explicit `None` handling.

Alleviation

[Mantle Network , 03/24/2026]: The issue has been fixed in this PR. <https://github.com/mantle-xyz/revm/pull/25>

[CertiK , 03/25/2026]: The team heeded the advice and resolved the issue in revm PR [25](#) by using fallible bytecode decoding (`Bytecode::new_raw_checked`) in AlloyDB ingestion and removing panic paths from malformed EIP-7702 code handling and block-hash lookup.

MAN-06 | Arsia Activation Path Lacks Runtime Verification That All Upgrade Transactions Succeeded

Category	Severity	Location	Status
Volatile Code, Inconsistency	● Informational	op-node/rollup/derive/arsia_upgrade_transactions.go (base-mantle-v2): 49~178; op-node/rollup/derive/attributes.go (base-mantle-v2): 124~149	● Acknowledged

Description

The derivation activation path deterministically injects Arsia upgrade transactions into payload attributes, but it does not include a runtime verification step that confirms all upgrade transactions executed successfully and that post-upgrade state is fully consistent. In a scenario where one activation transaction reverts while others succeed, the chain can progress in a partially applied upgrade state that is still deterministic but operationally fragile. This gap is primarily a fork-operations safety issue rather than a direct external exploit, yet it can increase troubleshooting complexity and recovery risk if activation does not land exactly as intended.

Recommendation

We recommend considering the addition of a minimal post-activation verification procedure that checks all Arsia upgrade transaction receipts are successful and validates key post-state invariants (including expected proxy implementation slots and `GasPriceOracle.isArsia == true`). We also recommend wiring this as an explicit operator-facing health check with clear alerting so partial activation can be detected and remediated immediately.

Alleviation

[Mantle Network , 03/18/2026]: Issue acknowledged. we would manually verify validities of all upgrade transactions once arsia is activated.

[CertiK , 03/18/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement. Moreover, client clarified that they will verify manually after the arsia activation.

MAN-08 | Unclear Panic Message

Category	Severity	Location	Status
Inconsistency	● Informational	op-chain-ops/genesis/mantle_config.go (base-mantle-v2): 77	● Resolved

Description

The function `SolidityMantleForkNumber()` evaluates the Mantle fork active at genesis and returns its Solidity enum index. It iterates over Mantle forks from highest (Arsia) down to Everest and returns the index of the first fork whose activation time is at genesis (offset 0). However, if none of the Mantle fork is active at genesis a panic is generated with the error message `panic("should never reach here")`, which is unclear and hard to debug.

Recommendation

We recommend replacing the generic panic with a descriptive message, clearly stating the invalid configuration.

Alleviation

[Mantle Network , 02/24/2026]: Issue acknowledged. Changes have been reflected in the commit hash:
<https://github.com/mantlenetworkio/mantle-v2/commit/e40c8d1d06827a0e1a602e70ba2b830789b5c6a9>

[CertiK , 02/25/2026]: The team heeded the advice and resolved the issue by adding a descriptive message in the panic in commit [e40c8d1d06827a0e1a602e70ba2b830789b5c6a9](https://github.com/mantlenetworkio/mantle-v2/commit/e40c8d1d06827a0e1a602e70ba2b830789b5c6a9)

MAN-09 | Usage Of Deprecated Types

Category	Severity	Location	Status
Volatile Code	● Informational	op-service/hsm/hsm_signer.go (base-mantle-v2): 77, 81	● Resolved

Description

At the pointed locations, there is the usage of deprecated code.

Recommendation

We recommend always using the updated version of third party dependencies and ensuring no deprecated function is used.

Alleviation

[Mantle Network , 03/03/2026]: Issue acknowledged. Changes have been reflected in the commit hash:

<https://github.com/mantlenetworkio/mantle-v2/commit/7938915128e40075b0cb96a09936f5c0ec346be4>

[Mantle Network , 03/03/2026]: The team heeded the advice and resolved the issue by updating the dependencies in commit [7938915128e40075b0cb96a09936f5c0ec346be4](https://github.com/mantlenetworkio/mantle-v2/commit/7938915128e40075b0cb96a09936f5c0ec346be4).

MAN-10 | Commented Out Code

Category	Severity	Location	Status
Coding Style	● Informational	op-chain-ops/genesis/config.go (base-mantle-v2): 220~231; op-node/rollup/derive/deposit_log.go (base-mantle-v2): 24~33; packages/contracts-bedrock/src/L2/GasPriceOracle.sol (base-mantle-v2): 5; packages/contracts-bedrock/src/L2/L1Block.sol (base-mantle-v2): 10; core/state_processor.go (base-op-geth): 23	● Resolved

Description

The highlighted line contains commented-out codes.

Recommendation

We recommend evaluating the usefulness of the commented-out code and eventually removing it to avoid any type of confusion from the codebase's readers.

Alleviation

[Mantle Network , 03/12/2026]:

Issue acknowledged. Changes have been reflected in the commit hash: <https://github.com/mantlenetworkio/mantle-v2/commit/b2b3eec4f21070018f7f0902d0ac3f8388a03a9b> <https://github.com/mantlenetworkio/op-geth/commit/89d197ec687bbfc0a1d9e1ca0a44ca64e795738>

[CertiK , 03/12/2026]:

The team heeded the advice and resolved the issue by cleaning up the commented-out and unused code, and clarifying the remaining configuration check comment, in commits [b2b3eec](#) and [89d197e](#).

MAN-12 | Deposit Log Parser Uses Hardcoded Byte Layout Arithmetic With Stale Inline Spec Comments

Category	Severity	Location	Status
Coding Style	● Informational	op-node/rollup/derive/deposit_log.go (base-mantle-v2): 57~59, 102~107, 154~158, 264~266	● Acknowledged

Description

The deposit log parser and marshaler rely on repeated hardcoded byte-width arithmetic (for example $32+32+32+8+1$ and $32+32+32+32+8+1$) and manual offset increments to decode and encode opaqueData. The current behavior appears consistent with the portal contract and mirrored client logic, but the implementation is maintenance-fragile because the layout assumptions are duplicated and partially documented with stale wording in comments. If a future protocol/layout update changes one side (contract emission, parser, or marshaler) without updating all mirrored assumptions, the system could drift into cross-client parsing inconsistencies that are difficult to detect early, especially near version transitions.

Recommendation

We recommend centralizing the opaque-data layout into explicit named constants/struct-level schema definitions (shared across parser and marshaler paths), and updating inline comments to exactly match the current versioned field order. We also recommend adding a version-aware round-trip conformance suite that asserts parser/marshaler compatibility against canonical contract-encoded vectors (including boundary lengths and mixed-version cases), so any future layout drift fails fast during CI.

Alleviation

[Mantle Network , 03/18/2026]: Issue acknowledged. I won't make any changes for the current version.

[CertiK , 03/18/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

MAN-13 | Unbounded `StorageKey` Decode Can Increase Memory Pressure On Proof Parsing Paths

Category	Severity	Location	Status
Denial of Service	● Informational	op-service/eth/types.go (base-mantle-v2): 824–845	● Acknowledged

Description

The `StorageKey` text unmarshal path currently accepts arbitrary-length hex input and decodes it without an explicit size bound. In practical usage this type is consumed by proof-related JSON-RPC flows where requested keys are expected to be 32-byte hashes, and later checks validate that response keys match requested `common.Hash` entries. That downstream check prevents silent semantic misuse, but the allocation/decode step still happens first, so unusually large inputs can still create avoidable memory pressure before rejection.

Recommendation

We recommend adding an explicit upper bound in `StorageKey.UnmarshalText` for RPC-facing decode paths (or introducing a strict 32-byte key variant where protocol semantics require fixed-size keys), and adding negative tests that exercise oversized key inputs to confirm fail-fast behavior.

Alleviation

[Mantle Network , 03/18/2026]: Issue acknowledged. I won't make any changes for the current version.

[Certik , 03/18/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

MAN-20 | NatSpec Semver Mismatch In OperatorFeeVault

Category	Severity	Location	Status
Inconsistency	● Informational	packages/contracts-bedrock/src/L2/OperatorFeeVault.sol (base-mantle-v2): 13~17	● Resolved

Description

The contract `OperatorFeeVault.sol` includes a NatSpec tag `@custom:semver 1.0.0` but the constructor instantiates `Semver(1, 1, 0)`. This mismatch does not affect runtime behavior but can create confusion in release notes, automated tooling, or developer understanding.

```
11:17:packages/contracts-bedrock/src/L2/OperatorFeeVault.sol
contract OperatorFeeVault is FeeVault, Semver {
    /**
     * @custom:semver 1.0.0
     *
     * @param _recipient Address that will receive the accumulated fees.
     */
    constructor(address _recipient) FeeVault(_recipient, 10 ether) Semver(1, 1, 0) {
    }
}
```

Recommendation

We recommend aligning the NatSpec `@custom:semver` with the constructor `Semver(...)`, or adjust the constructor semver if that is the intended authoritative value.

Alleviation

[Mantle Network , 03/03/2026]: Issue acknowledged. Changes have been reflected in the commit hash:
<https://github.com/mantlenetworkio/mantle-v2/commit/b510923f9927d7cf1480fc57c75154236468c8b5>

[Mantle Network , 03/03/2026]: The team heeded the advice and resolved the issue by adjusting the constructor semver to be aligned with the NatSpec `@custom:semver` in commit [b510923f9927d7cf1480fc57c75154236468c8b5](https://github.com/mantlenetworkio/mantle-v2/commit/b510923f9927d7cf1480fc57c75154236468c8b5)

MAN-21 | Unused Return Value In DeriveL1GasInfoMantle

Category	Severity	Location	Status
Coding Style	● Informational	core/state_processor.go (base-op-geth): 213; core/types/rollup_core.go (base-op-geth): 479~482	● Resolved

Description

`DeriveL1GasInfoMantle` returns five values, including `tokenRatio`, but the only in-repo callsite in `state_processor` ignores the fifth return value and uses a separately captured pre-execution `tokenRatio` instead. This behavior may introduce API ambiguity and increases maintenance risk as future refactors may incorrectly assume the returned `tokenRatio` is the authoritative value for receipt/fee accounting in the same flow.

Recommendation

We recommend simplifying the helper API to reflect real usage by removing the unused fifth return value from `DeriveL1GasInfoMantle`. Alternatively we recommend the dev team to clarify the design choice rational.

Alleviation

[Mantle Network , 03/18/2026]: Issue acknowledged. Changes have been reflected in the commit hash: <https://github.com/mantlenetworkio/op-geth/commit/cd179232179241dddb8ec3858610d861375576b1>

[CertiK, 03/18/2026]: The team heeded the advice and resolved the issue by removing the unused fifth return value in commit [cd179232179241dddb8ec3858610d861375576b1](https://github.com/mantlenetworkio/op-geth/commit/cd179232179241dddb8ec3858610d861375576b1).

MAN-41 | Mantle Arsia DA Footprint Accounting Can Panic On Uninitialized `BlobGasUsed`

Category	Severity	Location	Status
Coding Issue	● Informational	miner/worker.go (base-op-geth): 355~363, 467~474, 646~652, 860~864	● Resolved

Description

The miner path for Jovian DA footprint accounting assumes that `header.BlobGasUsed` is always non-nil whenever `IsMantleArsia` is true. In `prepareWork`, `header.BlobGasUsed` is only initialized inside the Cancun fork guard (`IsCancun(header.Number, header.Time)`), but later paths in `commitBlobTransaction`, `commitTransactions`, and `commitFIFOTransactions` unconditionally dereference `*env.header.BlobGasUsed` when `IsMantleArsia(env.header.Time)` is true. If a future configuration activates Mantle Arsia while Cancun is disabled or not yet active for the same blocks, `header.BlobGasUsed` remains nil and these dereferences will panic, causing miner crashes on affected blocks.

Recommendation

We recommend making the Mantle Arsia DA accounting paths robust to configuration changes by either:

- initializing `header.BlobGasUsed` to zero whenever `IsMantleArsia` is true (even if Cancun is disabled for that block)
- guarding all uses of `*header.BlobGasUsed` with explicit nil checks and returning a controlled error instead of panicking.

Alleviation

[Mantle Network, 03/10/2026]: Acknowledged. Cancun is a prerequisite for MantleArsia in the OP-Stack fork sequence and Arsia builds on Cancun. All chain configurations (mainnet, sepolia, devnet) activate Cancun at genesis (`CancunTime = 0`), and it is architecturally impossible to reach an Arsia block without Cancun already being active. The DA accounting intentionally reuses the Cancun-initialized `BlobGasUsed` field by design.

We consider the current code correct for all supported configurations and do not plan to change it. Adding a nil guard would imply that Arsia-without-Cancun is a supported configuration, which it is not.

[CertiK, 03/10/2026]: The team provided evidence that the panic should not be reachable in any supported configuration: Cancun is a strict prerequisite for MantleArsia, all Mantle networks activate Cancun at genesis, and Arsia is never enabled without Cancun. In practice, the code relies on this fork-order invariant and intentionally treats a missing `BlobGasUsed` as a fatal integrity violation rather than a recoverable condition.

Given this architectural constraint and the explicit decision not to support “Arsia without Cancun” configurations, we consider

the finding resolved by design.

APPENDIX | MANTLE NETWORK

Audit Scope

mantlenetworkio/mantle-v2

 op-node/rollup/derive/arsia_upgrade_transactions.go

 op-node/rollup/derive/attributes.go

 op-node/rollup/derive/deposit_log.go

 op-service/eth/types.go

 op-batcher/batcher/driver.go

 op-batcher/batcher/service.go

 op-batcher/batcher/tx_data.go

 op-chain-ops/genesis/mantle_config.go

 op-chain-ops/genesis/config.go

 op-node/cmd/genesis/cmd.go

 op-node/config/backup_sync_rpc.go

 op-node/flags/flags.go

 op-node/node/node.go

 op-node/rollup/derive/data_source.go

 op-node/rollup/derive/mantle_blob_source.go

 op-node/rollup/mantle_types.go

 op-service/crypto/signature.go

 op-service/hsm/hsm_signer.go

 op-service/sources/backup_sync_client.go

mantlenetworkio/mantle-v2

 packages/contracts-bedrock/src/L1/SystemConfig.sol packages/contracts-bedrock/src/L2/GasPriceOracle.sol packages/contracts-bedrock/src/L2/L1Block.sol packages/contracts-bedrock/src/L2/OperatorFeeVault.sol op-chain-ops/addresses/contracts.go op-chain-ops/addresses/mantle_contracts.go op-core/forks/mantle_forks.go op-node/cmd/main.go op-node/config/config.go op-node/node/backup_sync.go op-node/p2p/gossip.go op-node/p2p/sync.go op-node/rollup/derive/batches.go op-node/rollup/derive/l1_block_info.go op-node/rollup/derive/mantle_system_config.go op-node/rollup/derive/skadi_upgrade_transactions.go op-node/rollup/derive/system_config.go op-node/rollup/attributes/engine_consolidate.go op-node/rollup/types.go op-node/mantle_service.go op-node/service.go op-program/client/l2/engineapi/block_processor.go

mantlenetworkio/mantle-v2

 op-program/client/mpt/db.go op-service/flags/mantle_flags.go op-service/flags/flags.go op-service/txmgr/cli.go

mantle-xyz/revm

 crates/op-revm/src/handler.rs crates/op-revm/src/transaction/bvm_eth.rs crates/handler/src/validation.rs crates/op-revm/src/constants.rs crates/op-revm/src/l1block.rs crates/op-revm/src/precompiles.rs crates/op-revm/src/spec.rs crates/op-revm/src/transaction/abstraction.rs

mantle-xyz/kona

 crates/protocol/protocol/src/deposits.rs crates/protocol/derive/src/attributes/stateful.rs crates/proof/executor/src/builder/assemble.rs crates/protocol/genesis/src/rollup.rs crates/protocol/protocol/src/batch/single.rs crates/protocol/derive/src/sources/mantle_blob.rs crates/protocol/derive/src/sources/mantle_ethereum.rs

mantle-xyz/kona

 bin/client/src/fpvm_evm/precompiles/provider.rs bin/host/src/backend/online.rs crates/proof/executor/src/builder/core.rs crates/proof/executor/src/builder/env.rs crates/proof/proof-interop/src/consolidation.rs crates/protocol/genesis/src/chain/config.rs crates/protocol/genesis/src/chain/mantle_hardfork.rs crates/protocol/genesis/src/chain/mod.rs crates/protocol/genesis/src/lib.rs crates/protocol/genesis/src/params.rs crates/protocol/genesis/src/genesis.rs crates/protocol/genesis/src/system/config.rs crates/protocol/genesis/src/system/errors.rs crates/protocol/genesis/src/system/kind.rs crates/protocol/genesis/src/system/log.rs crates/protocol/genesis/src/system/mod.rs crates/protocol/genesis/src/system/update.rs crates/protocol/genesis/src/updates/base_fee.rs crates/protocol/genesis/src/updates/mod.rs crates/protocol/hardforks/src/arsia.rs crates/protocol/hardforks/src/ecotone.rs crates/protocol/hardforks/src/fjord.rs

mantle-xyz/kona

-  crates/protocol/hardforks/src/interop.rs
-  crates/protocol/hardforks/src/isthmus.rs
-  crates/protocol/hardforks/src/jovian.rs
-  crates/protocol/hardforks/src/lib.rs
-  crates/protocol/hardforks/src/mantle_forks.rs
-  crates/protocol/protocol/src/info/jovian.rs
-  crates/protocol/protocol/src/info/variant.rs
-  crates/protocol/derive/src/sources/mod.rs
-  crates/protocol/derive/src/stages/batch/batch_queue.rs
-  crates/protocol/derive/src/lib.rs

mantlenetworkio/op-geth

-  core/state_processor.go
-  core/state_transition.go
-  core/txpool/validation.go
-  core/types/rollup_cost.go
-  consensus/misc/eip1559/eip1559.go
-  eth/catalyst/api.go
-  internal/ethapi/api.go
-  miner/preconf_checker.go
-  miner/worker.go
-  core/block_validator.go

mantlenetworkio/op-geth

-  core/blockchain.go
-  core/evm.go
-  core/genesis.go
-  core/rawdb/freezer_memory.go
-  core/rawdb/accessors_history.go
-  core/rawdb/ancient_scheme.go
-  core/rawdb/freezer_table.go
-  core/rawdb/schema.go
-  core/txpool/legacypool/legacypool.go
-  core/txpool/legacypool/queue.go
-  core/txpool/rollup.go
-  core/types/hashing.go
-  core/types/receipt.go
-  core/types/receipt_opstack.go
-  core/vm/evm.go
-  consensus/misc/eip1559/eip1559_optimism.go
-  eth/catalyst/api_optimism.go
-  eth/filters/filter.go
-  eth/filters/filter_system.go
-  eth/protocols/eth/handlers.go
-  eth/tracers/native/keccak256_preimage.go
-  internal/ethapi/simulate.go

mantlenetworkio/op-geth

-  miner/payload_building.go
-  p2p/discover/lookup.go
-  superchain/chain.go
-  superchain/superchain.go
-  superchain/types.go
-  trie/bytetest.go
-  trie/list_hasher.go
-  trie/stacktrie.go
-  triedb/pathdb/history.go
-  triedb/pathdb/history_indexer.go
-  triedb/pathdb/history_trienode.go

Finding Categories

Categories	Description
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Denial of Service	Denial of Service findings indicate that an attacker may prevent the program from operating correctly or responding to legitimate requests.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.

Categories	Description
Incorrect Calculation	Incorrect Calculation findings are about issues in numeric computation such as rounding errors, overflows, out-of-bounds and any computation that is not intended.
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

Mantle Network Security Assessment | CertiK Assessed on Mar 26th, 2026 | Copyright © CertiK

